

Introduction to Blockchain and Smart Contracts

Lecture 11: Smart Contract Vulnerabilities, Tokens

Subodh Sharma | IIT Delhi | March 12, 2026



Oracle Price Manipulation

Impact

- In 2022, over **\$403.2 million** were stolen in **DeFi** as a result of over 40 **price oracle manipulation attacks**.
- Oracles enable smart contracts to execute actions **based on inputs and outputs** from the real world or from other smart contracts and chains.
 - Data is brought into blockchains from external sources through oracles
- Examples of Oracles: **Chainlink & Uniswap** (DEX)
- Most oracle manipulation attacks through **Flash Loans**
 - **Flash loan:** borrowing without the need for collateral

Oracle Price Manipulation

Price Oracle Vulnerability

- **Price Oracle:** Mechanism that fetches real-world price data (eg: ETH/USD, BTC/USD) and makes it available to smart contracts
- If the oracle relies **on a single exchange** or API, that source becomes **single point of failure**
 - An attacker can manipulate the price on that specific, low-liquidity exchange
- **Flash loan attacks:** **All within a single atomic transaction** — Borrow a large amount of asset without collateral —> use it to temporarily manipulate the price of a token —> exploit the DeFi protocol on the manipulated price —> repay the flash loan
- **Centralised oracles:** Introduces a trust assumption; can be malicious

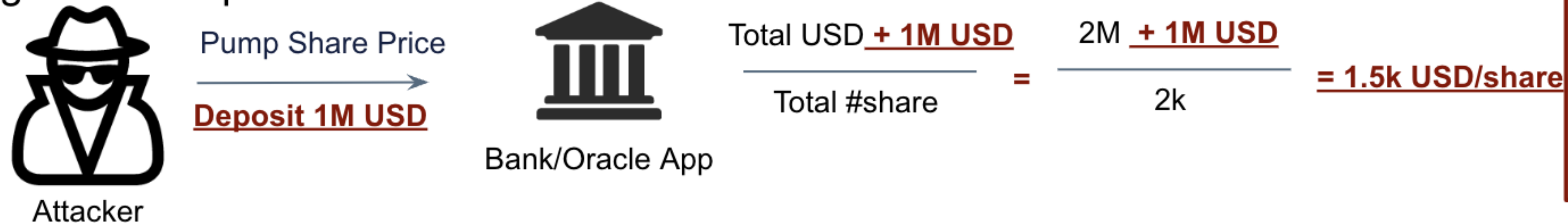
Price Oracle Manipulation Attack (POMA)



Stage #1: Buy

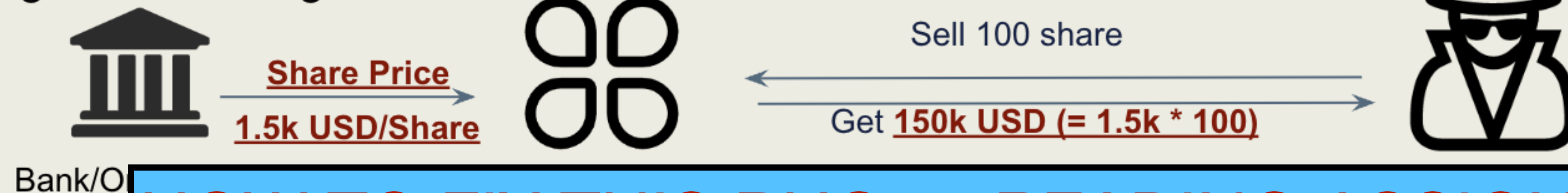


Stage #2: Manipulate



Attacker earns profit of 50k USD

Stage #3: Arbitrage



HOW TO FIX THIS BUG — READING ASSIGNMENT

Delegate Call

- **Delegate Call:** Contract A runs code from Contract B **but in A's own context**
 - That is: the storage, current address, and balance remains of the calling contract
 - `msg.sender` and `msg.value` do not change
- Let us take an example to understand this more!
- Why are they useful?
 - Proxy upgrade patterns
 - Library-style reusable code

Delegate Call

```
pragma solidity ^0.8.0;

contract Logic {
    uint256 public x;

    function setX(uint256 v) external {
        x = v;
    }
}
```

Callee

```
contract Caller {
    uint256 public x;
    address public logic;

    constructor(address _logic) {
        logic = _logic;
    }

    function setViaDelegatecall(uint256 v) external {
        (bool ok, ) = logic.delegatecall(
            abi.encodeWithSignature("setX(uint256)", v)
        );
        require(ok, "delegatecall failed");
    }
}
```

Caller

Delegate Call Risk

- **Key idea:** One could have arbitrary code execution over your contract's storage
 - Attack instance: An attacker can change the owner of the contract

```
contract Evil {  
    address public owner;    //  
  
    function pwn() external {  
        owner = msg.sender;  
    }  
}
```

- **Fixes:**
 - Never delegate to user-controlled address
 - Select from whitelisted implementations only

```
BadWallet.exec(address(evil), abi.encodeWithSignature("pwn()"))
```


Tokens

- How are they used?
 - Currency, Resource, Asset, Equity, Voting, Identity, Utility, etc.
- Tokens are mostly *fungible*
 - Non-fungible tokens also exist: such as NFTs
- ERC20 Token Standard
 - Defines a common interface for contracts implementing a token

ERC20 Token

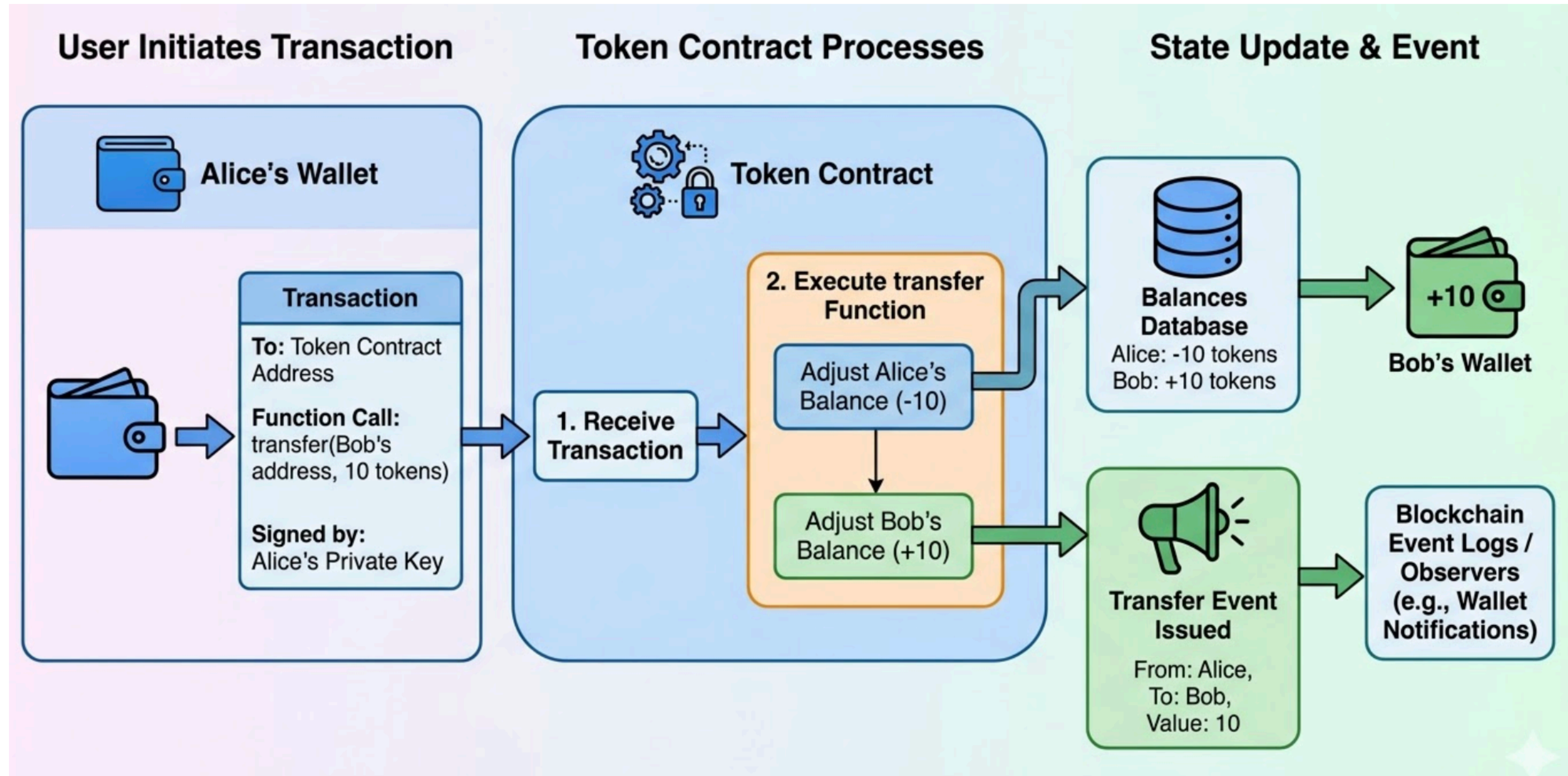
- ERC20 compliant token must at least provide following functions and events:

- totalSupply
- balanceOf(addr)
- transfer
- transferFrom
- approve
- allowance

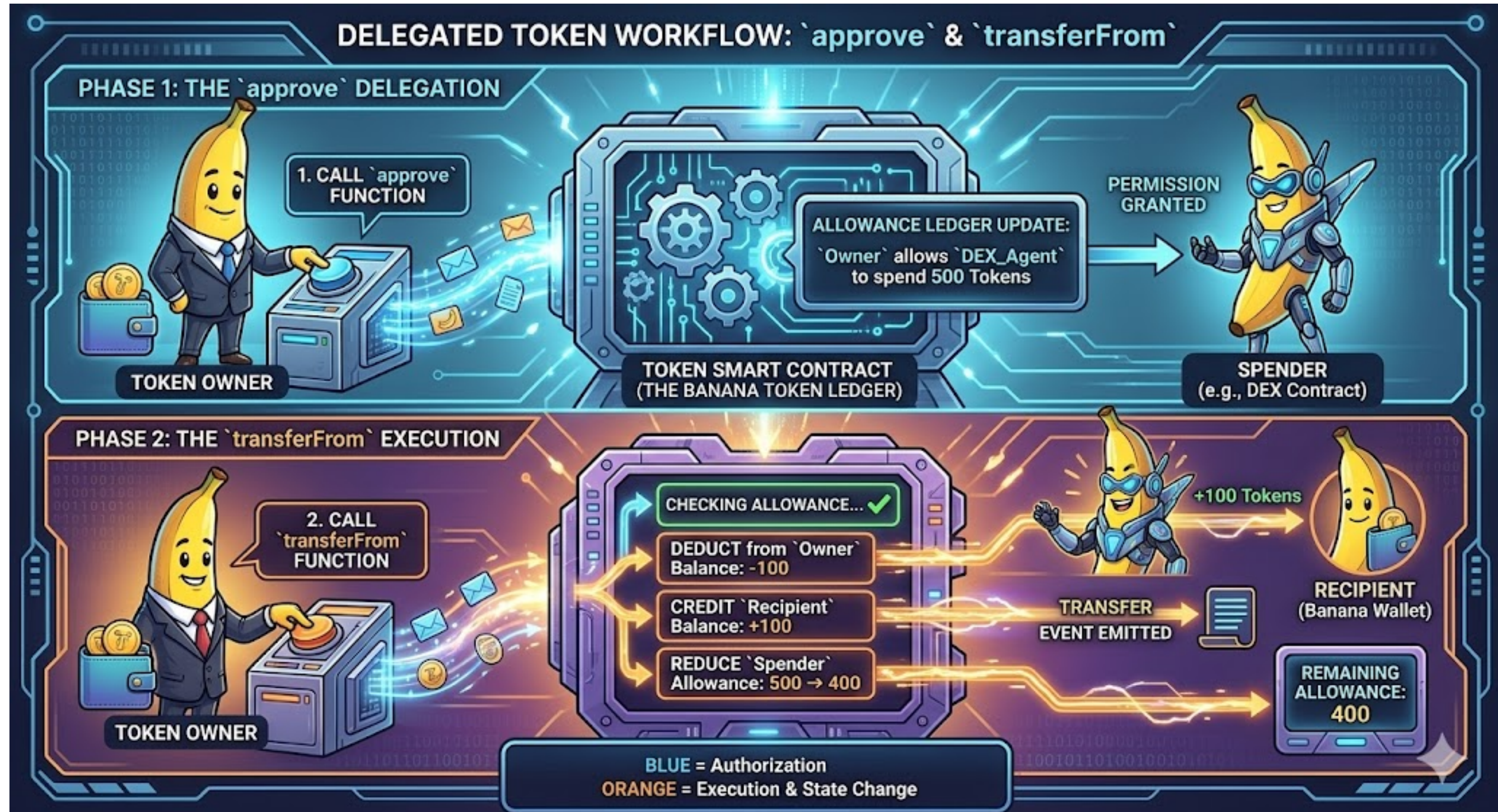
```
contract ERC20 {  
    function totalSupply() constant returns (uint theTotalSupply);  
    function balanceOf(address _owner) constant returns (uint balance);  
    function transfer(address _to, uint _value) returns (bool success);  
    function transferFrom(address _from, address _to, uint _value) returns  
        (bool success);  
    function approve(address _spender, uint _value) returns (bool success);  
    function allowance(address _owner, address _spender) constant returns  
        (uint remaining);  
    event Transfer(address indexed _from, address indexed _to, uint _value);  
    event Approval(address indexed _owner, address indexed _spender, uint _value);  
}
```

- EVENTS: Approval, ...

ERC20 Token



ERC20 Token



ERC20 Data Structure

- `mapping (address => uint256) balances;`
 - Internal table by the owner to track the ownership of tokens
- `mapping (address => mapping (address => uint256)) public allowed;`